



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Project

Interim Report

25% Individual Coursework

Submission: Final Submission

AY 2025 2026

Academic Semester: Autumn Semester 2025

Credit: 30 Credit Year Long Module

Student Name: Samriddhi Poudel

London Met ID: 23047345

College ID: NP01NT4A230037

Assignment Due Date: Tuesday, December 30, 2025

Assignment Submission Date: Tuesday, December 30, 2025

Internal Supervisor: Anuj Shilpakar

External Supervisor: Rabindra Khadka

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Acknowledgement

I would like to express my sincere gratitude to my internal supervisor, Mr. Anuj Shilpakar, and my external supervisor, Mr. Rabindra Khadka, for their invaluable guidance, encouragement, and support throughout the development of this project. Their expertise and insightful feedback have helped me stay focused and continuously improve the quality of my work.

I am also grateful to my faculty, colleagues, and friends for their continuous motivation and assistance during this journey. Finally, I thank my family for their unwavering support and understanding, which made completing this project possible.

Abstract

APIs (Application Programming Interfaces) are requisite to contemporary software systems, facilitating seamless integration, automation, and enhanced user experiences. However, their accessibility and openness expose them to high levels of security breaches such as failed authentication, broken authorization, and security misconfigurations. The increasing rate of API-related attacks, along with inadequate monitoring and control, necessitates a proactive approach to security measures.

This interim report documents the progress made in developing an **API Security Tester and Vulnerability Scanner** designed to improve organizational API security posture. The tool detects typical vulnerabilities, tracks APIs for suspicious behavior, creates warnings, and generates detailed reports. The system provides end-to-end protection for RESTful and third-party APIs by incorporating automated security testing, real-time monitoring capabilities, and interactive dashboards.

Following the Agile Incremental Framework, the project has been developed in functional modules through continuous testing, refinement, and adaptation. After two weeks of development, the system successfully implements 10 comprehensive security tests covering major OWASP API Security Top 10 categories, including broken authentication, injection attacks, excessive data exposure, and rate limiting issues.

The current prototype demonstrates the feasibility of automated API security testing through a user-friendly web-based application. With both technical and non-technical users in mind, the tool provides vulnerability identification, automated monitoring foundations, and clear reporting mechanisms. This interim report details the progress achieved, methodologies applied, challenges overcome, and the roadmap for completing the remaining advanced features.

Table of Contents

1. Introduction.....	1
1.1. Introduction to topic	1
1.2. Problem Scenario	3
1.3. Project as a solution	4
1.4. Aim and Objectives.....	5
1.4.1. Aim.....	5
1.4.2. Objectives.....	5
1.5. Structure of the report.....	6
1.5.1. Background.....	6
1.5.2. Development	6
1.5.3. Analysis of progress	6
1.5.4. Further Work.....	6
2. Background	7
2.1. Client's Description and Requirements.....	7
2.1.1. Client's Description.....	7
2.1.2. Client's Requirements	8
2.2. Understanding the project	9
2.2.1. Project Description	9
2.2.2. Project Deliverables	9
2.3. Similar Projects Review.....	10
2.3.1. Comparison table	13
2.3.2. Analysis and conclusion of comparison.....	14
3. Development Methodology	15
3.1. Considered Methodology.....	15
3.1.1. Spiral Methodology.....	15
3.1.2. Prototype Methodology	15
3.2. Selected Methodology	15

3.2.1.	Agile Methodology.....	15
3.3.	Work Breakdown Structure	16
3.3.1.	WBS for the project	16
3.4.	Gantt Chart.....	17
3.4.1.	Previous Gantt Chart.....	17
3.4.2.	Updated Gantt Chart	18
3.5.	Milestone	19
4.	Development Till date	19
4.1.	System Architecture Diagram.....	19
4.2.	Use Case Diagram	20
4.3.	Flowchart.....	20
4.4.	Wireframe.....	22
4.5.	UI Design	23
4.6.	Development status with evidence.....	23
4.6.1.	Development status.....	23
4.6.2.	Evidence	23
5.	Analysis of Progress	28
5.1.	Progress Review.....	28
5.1.1.	Current Scenario of Progress	28
5.1.2.	Progress Timeline	28
5.1.3.	Action Plan.....	29
6.	Further Work	30
7.	Conclusion.....	32
8.	References	33
9.	Appendix.....	35
9.1.	Appendix 1: Problem Scenario.....	35

9.2.	Appendix 2: Client's Requirements	36
9.3.	Appendix 3: Spiral Methodology	38
9.4.	Appendix 4: Prototype Methodology	39
9.5.	Appendix 5: Agile Methodology.....	40
9.5.1.	Incremental Framework	41
	Phases of the Incremental Framework.....	41
9.6.	Appendix 6: Milestones	43
1.1.1.	Progress of milestones.....	44
9.7.	Appendix 7: Findings of Pre-Survey.....	46

Table of figures

Figure 1 Estimated increase in API attacks (Kong, 2023)	1
Figure 2 API Threat Landscape (escape, 2024)	3
Figure 3 OWASP Zed Attack Proxy (ZAP)	10
Figure 4 Postman with Newman (Collection Runner)	11
Figure 5 Burp Suite Professional	12
Figure 6 WBS for the project.....	16
Figure 7 Gantt Chart.....	17
Figure 8 Updated gantt chart	18
Figure 9 System Architecture Diagram.....	19
Figure 10 Use Case Diagram.....	20
Figure 11 Flowchart.....	21
Figure 12 Wireframe.....	22
Figure 13 UI Design.....	23
Figure 14 project structure in VS Code.....	24
Figure 15 Flask backend server running	25
Figure 16 API security tester scanning api	26
Figure 17 Scan, vulnerability tables and sample data.....	27
Figure 18 API endpoints stored in the database	27
Figure 19 Milestone	43
Figure 20 Pre-survey finding 1	46
Figure 21 Pre-survey finding 2.....	46
Figure 22 Pre-survey finding 3.....	47
Figure 23 Pre-survey finding 4.....	47
Figure 24 Pre-survey finding 5.....	48
Figure 25 Pre-survey finding 6.....	48
Figure 26 Pre-survey finding 7	49
Figure 27 Pre-survey finding 8.....	49
Figure 28 Pre-survey finding 9.....	50
Figure 29 Pre-survey finding 10.....	50
Figure 30 Pre-survey finding 11.....	51
Figure 31 Pre-survey finding 12.....	51
Figure 32 Pre-survey finding 13.....	52
Figure 33 Pre-survey finding 14.....	52
Figure 34 Pre-survey finding 15.....	53
Figure 35 Pre-survey finding 16.....	53

Table of tables

Table 1 Comparision of similar tables.....	13
Table 2 Progress table of milestones	45

1. Introduction

1.1. Introduction to topic

The current software systems are based on APIs (Application Programming Interfaces) as the glue between applications and services. They promote data flow, automation, combine microservices and cloud-native platforms, enhance operations and innovate. APIs are now utilized by over 85 percent of the businesses to deliver smooth experiences to their customers and attain digital transformation (Kukushkina, 2025).

Although this is of critical importance, APIs are susceptible because of the openness and accessibility. T-Mobile (2022), LinkedIn (2021), Facebook (2019), Strava (2018), and Equifax (2017) are just but a few examples that illustrate the fact that even highly developed API gateways cannot resist attacks (Penelope Todd, 2025).

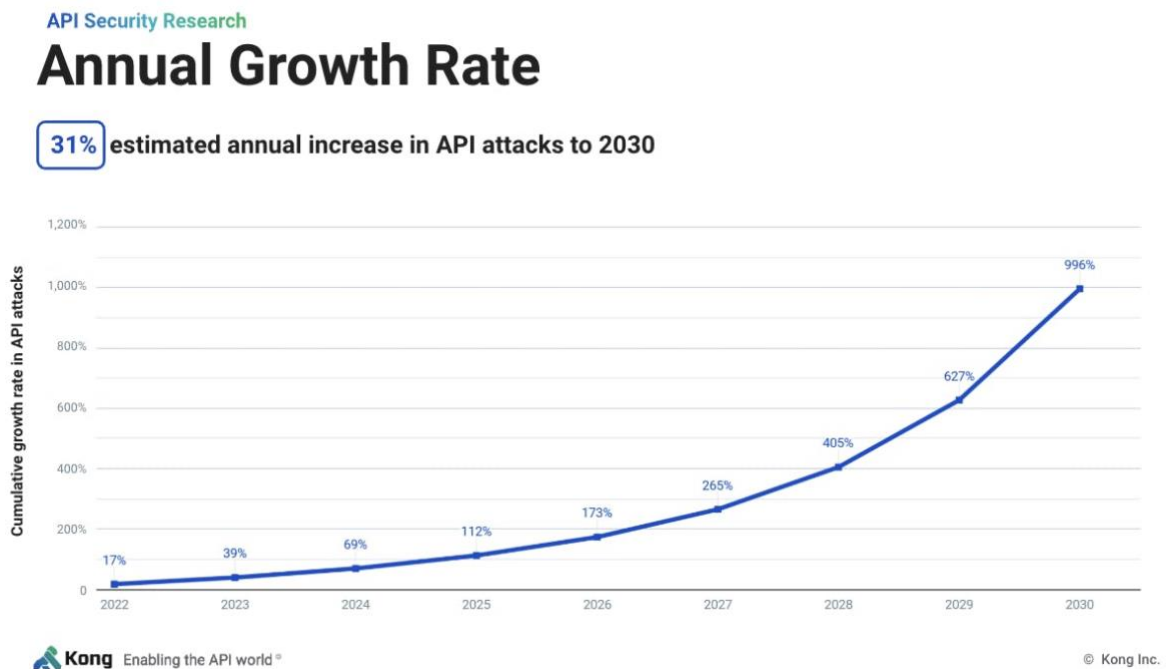


Figure 1 Estimated increase in API attacks (Kong, 2023)

The key steps to ensure API security are authentication, authorization, encryption, input validation, rate limiting, logging, monitoring, and secure coding practices that could minimize unauthorized access and data breaches. The API security of more

complex IT ecosystems, such as microservices and third-party integrations need special measures in addition to usual web application security measures (Chinnasamy, 2023).

This report entails the developments taking place in the creation of an automated API Security Tester and Vulnerability Scanner that can help deal with these issues. The system has been built using iteration cycles in accordance with Agile Incremental Framework as suggested in the project proposal, with functional modules being delivered gradually during the first two weeks of development.

1.2. Problem Scenario

There are a lot of factors because APIs are highly vulnerable to security breaches. The common vulnerabilities are broken object-level authorization, failed authentication, broken function-level authorization and incomplete input validation among others. All of these vulnerabilities allow hackers to log into unauthorized services, steal confidential information, or even disrupt services (OWASP, 2023).

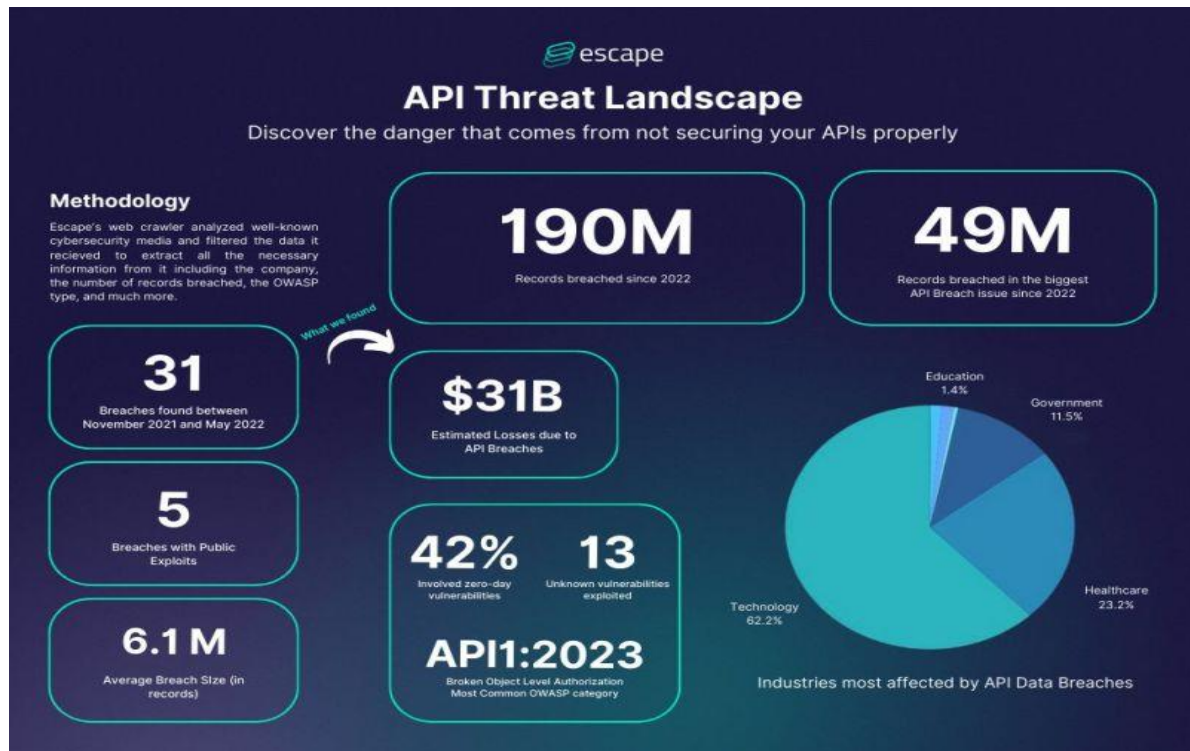


Figure 2 API Threat Landscape (escape, 2024)

Also, poor inventory management and documentation, unregulated third-party APIs, and unlimited usage of resources, increases the attack surface to a larger extent. Most organizations do not fully see their API endpoints and integrations, which provides attackers with some blind spots. The security challenge is increasing exponentially with the proliferation of APIs to enable the processes of the digital transformation (F5 Newsroom, 2025).

Read about Key Problem Areas Identified in [Appendix 1](#)

1.3. Project as a solution

To overcome such obstacles, this project creates API Security Tester and Vulnerability Scanner which provides a single platform to:

- Determine vulnerabilities including authentication failure, broken authorization, SSRF, and injection attacks, security misconfigurations.
- Keep an eye on APIs and follow abnormal behavioural patterns.
- Generate warnings and reports to mitigate potential threats proactively.

The project improves the organizational API security posture through automation of repetitive testing, ability to perform continuous monitoring, and extensive reporting of performance that minimizes the operational risk and increases availability of services.

1.4. Aim and Objectives

1.4.1. Aim

The project aims to design and deploy an all-in-one API Security Tester and Vulnerability Scanner that will secure APIs by detecting weaknesses, maintaining security standards, and providing real-time monitoring and notification frameworks.

1.4.2. Objectives

- Create a tool to assess APIs for prevalent vulnerabilities, such as broken authentication, authorization issues, SSRF, and security misconfigurations.
- Set up automated surveillance to identify unusual API behavior and possible attacks.
- Incorporate instant alerts and notifications for critical API vulnerabilities.
- Deliver user-friendly dashboards and reports to illustrate API security status for both technical and non-technical audiences.

1.5. Structure of the report

1.5.1. Background

The Background section will give a project context and justification. It describes the perspective of the client or problem-domain, specifies the scope of the API Security Tester and Vulnerability Scanner, and examines the concepts and similar solutions. This section is used to define the need of the project and makes it fit into the existing API security issues.

1.5.2. Development

The Development section includes the explanation of the methodology followed and the technical work performed in the interim period. It outlines the chosen Agile Incremental Framework, system architecture, implementation strategy, and important elements that have been developed up to date. In this section, it is shown how the planned methods have been put into practice.

1.5.3. Analysis of progress

Analysis of Progress section compares the work done with the initial project proposal. It shows progress tables, milestones reviews and time schedule comparisons to check the achievement of objectives. Critical reflection is also discussed by having challenges faced and the efficiency of mitigation measures.

1.5.4. Further Work

The Further Work section describes the rest of the actions to be undertaken to accomplish the project. It determines the future stages of development, testing processes, and final modifications, as well as a renewed schedule. This section will reflect forward planning and ensure the possibility of the project being completed within the stipulated schedule.

2. Background

2.1. Client's Description and Requirements

2.1.1. Client's Description

The project is a Final Year Project in CS6PO5NI of the London Metropolitan University under the supervision of Mr. Anuj Shilpakar as internal and Mr. Rabindra Khadka as external supervisor. The project would also solve a dire industry requirement as determined by literature review and conducted initial stakeholder analysis.

The main stakeholders of this API security tester and vulnerability scanner are:

Primary Users:

- **Software Development Teams:** Developers who use or develop third-party APIs, or create their own ones, will require addressing the issue of security compliance prior to deployment.
- **Security Engineers and DevSecOps Teams:** The specialists who ensure that API security is in posture at the organizational level and the vulnerability assessment is conducted on a regular basis.
- **Quality Assurance Teams:** QA engineers that have security testing as one of their testing procedures.
- **IT Security Managers:** Decision-makers who require detailed reports on the status of API security to manage risks.

Secondary Users:

- **Small to Medium Enterprises (SMEs):** Companies that do not have any specific security division, yet they must validate the API security.
- **Independent Developers:** Free-lancers and individual developers that develop API-driven apps.
- **Educational Facilities:** those students and researchers who analyse the concepts of API security and cybersecurity.

2.1.2. Client's Requirements

Through the analysis of industry best practices, some functional and non-functional requirements were found.

Read about the requirements in [Appendix 2](#)

2.2. Understanding the project

2.2.1. Project Description

The **API Security Tester and Vulnerability Scanner** is a comprehensive web-based application designed to automate the detection of security vulnerabilities in RESTful APIs. The system follows a three-tier architecture comprising a frontend user interface, backend REST API server, and MySQL database for persistent storage.

2.2.2. Project Deliverables

The application accepts API endpoints as input and executes several automated security tests aligned with the OWASP API Security Top 10 framework. Each test targets specific vulnerability categories:

- **Endpoint Reachability Test:** Validates API availability and response status
- **HTTPS Enforcement Test:** Ensures encrypted communication channels
- **HTTP Methods Security Check:** Identifies exposed dangerous methods (TRACE, DELETE)
- **Security Headers Analysis:** Validates protective HTTP headers (X-Frame-Options, Content-Security-Policy, X-Content-Type-Options)
- **Broken Authentication Check:** Tests for missing or weak authentication mechanisms
- **SQL Injection Detection:** Probes for database injection vulnerabilities using common payloads
- **XSS (Cross-Site Scripting) Detection:** Tests for unsanitized script injection
- **Rate Limiting Verification:** Assesses API throttling and brute-force protection
- **Excessive Data Exposure Check:** Scans responses for sensitive information leakage
- **SSRF (Server-Side Request Forgery) Detection:** Tests for internal network access vulnerabilities

2.3. Similar Projects Review

To guarantee that the API Security Tester offers distinctive benefits and includes top industry practices, a thorough examination of current API security and testing tools was performed. This review analyses four leading solutions from various categories: specialized security scanners, API development platforms, expert penetration testing tools, and testing frameworks. Every tool is assessed according to its characteristics, advantages, drawbacks, and significance to the goals of this project.

Project 1: OWASP Zed Attack Proxy (ZAP)



Figure 3 OWASP Zed Attack Proxy (ZAP)

OWASP Zed Attack Proxy (ZAP) is an open-source vulnerability testing tool, which is intended to locate vulnerabilities in web apps and RESTful APIs. It facilitates importation of API definitions, scanning of endpoints on frequent problems, including injection vulnerabilities, failed authentication, and the lack of security-related headers (Singh, 2021).

ZAP is good when you want to perform security checks but lacks continuous monitoring of the API security or easy-to-use dashboards that are not technical making it unsuitable as a complete API security posture tool.

Project 2: Postman with Newman (Collection Runner)

postmanlabs/ newman



Newman is a command-line collection runner for Postman

👤 105

Contributors



25k

Used by



7k

Stars



1k

Forks



Figure 4 Postman with Newman (Collection Runner)

Postman is a user-friendly tool of designing, documenting and manually testing APIs using request collections, environment and test scripts. Newman further makes this ability available to the command line, enabling Postman collections and their tests to be run in headless mode, outside of the Postman GUI, with Node.js or Docker (Postman , 2025).

Newman has extensive support of rich command-line options, uploading data files with test files, and supporting multiple reporters to generate machine-readable or human-readable test reports, and is therefore appropriate to be included in CI pipelines, including Jenkins or Travis CI. Nevertheless, this stack is rather functional and regression-oriented; it lacks in-depth and OWASP API Security Top 10-oriented security testing, centralized vulnerability history, and specific security dashboards, which are the primary goals of the proposed API Security Tester and Vulnerability Scanner.

Project 3: Burp Suite Professional



Figure 5 Burp Suite Professional

Burp Suite Professional is a commercial penetration testing platform based on a popular semi-automated and manual penetration security testing of web applications and APIs. It has some advanced functions including interception proxy, automatable scanning, fuzzing intruder and extensive support of exploration of intricate authentication and authorization paths (Portswigger, 2025).

Burp Suite is, however, not user-friendly and is more geared towards expert level penetration testers and needs a lot of expertise to be configured and results properly interpreted. It is not as convenient as a self-service, dashboard driven tool to provide continuous API security assurance by development and quality assurance teams and the cost of their license is high when compared to a low weight custom solution.

2.3.1. Comparison table

S.N	Features / Criteria	OWASP ZAP	Postman+Newman	Burp Suite Pro	API Security Tester
1.	Automated scan of REST APIs	Yes	Limited Scripts	Yes	Yes
2.	Focus on OWASP API Security Top 10	Partial	No(manual)	Partial	Yes
3.	Real-time monitoring dashboard	No	No	No	Yes
4.	Historical scan storage and reporting	Basic Logs	Limited	Reports only	Yes
5.	Ease of use for non-technical users	Medium	High	Low	High
6.	Designed specifically for API security posture	No	No	No	Yes

Table 1 Comparison of similar tables

2.3.2. Analysis and conclusion of comparison

Based on this comparison, it can be observed that current tools are oriented to generic web security (ZAP, Burp) or to functional API testing and team integration (Postman), whereas API-specific Top 10 are also structured in their coverage but cannot be used in ongoing monitoring. The proposed API Security Tester will provide a combination of automated tests of the OWASP API Top 10 based on the latest data, continuous historical storage, and user-friendly dashboards into one platform that is highly sought after by both technical and non-technical users.

The project eliminates the flaws of disjointed tooling and complex expert only interfaces found in the reviewed solutions by providing guided vulnerability checking, live visualization, and scheduled scanning through a single system.

3. Development Methodology

3.1. Considered Methodology

While planning the project, different methodologies were researched and explored. Some of the methodologies that were considered are given below:

3.1.1. [Spiral Methodology](#)

3.1.2. [Prototype Methodology](#)

3.2. Selected Methodology

3.2.1. Agile Methodology

Agile methodology is a flexible and iterative approach to software development that focuses on delivering working software quickly while adapting to changing requirements. It emphasizes collaboration, continuous feedback, and improvement rather than rigid planning and heavy documentation (Paulk, 2002).

Read more about agile methodology in [Appendix 3](#)

3.2.1.1. Incremental Framework

The Incremental Framework is a software development approach in which the system is developed and delivered in small, manageable increments. Each increment goes through the phases of requirement analysis, design, development, testing, and implementation. With every increment, new functionality is added until the complete system is achieved (geeksforgeeks, 2025).

[Click here](#) to learn more about incremental framework.

3.3. Work Breakdown Structure

A Work Breakdown Structure is a useful tool for managing projects. It helps people in charge of projects to break them down into pieces that are easy to handle. The Work Breakdown Structure starts with what the project's supposed to achieve and then breaks it down into things that need to be done and smaller tasks. This shows how each smaller task helps to get to the result of the Work Breakdown Structure (Raeburn, 2025).

The Work Breakdown Structure is, about making sure everyone knows what they are doing to help the project succeed. WBS helps project managers organize work hierarchically, making complex projects easier to understand, plan, and track. It provides a clear overview of project scope, deliverables, dependencies, and responsibilities (Raeburn, 2025).

3.3.1. WBS for the project

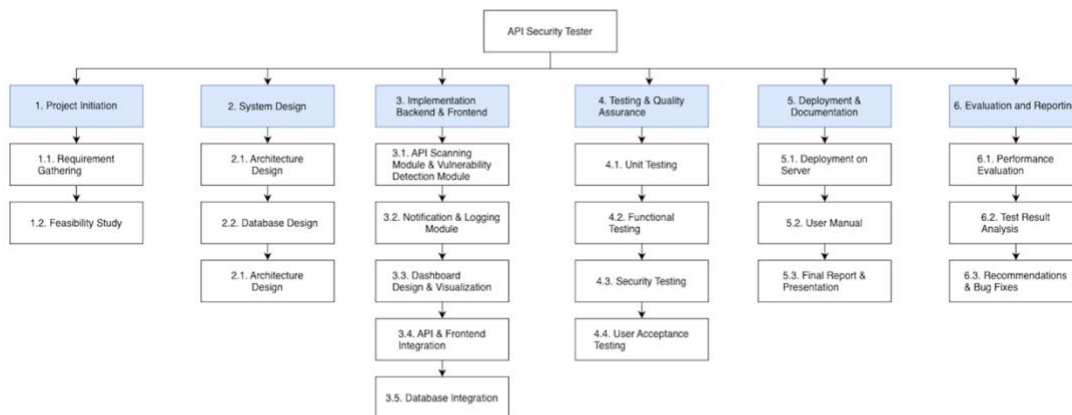


Figure 6 WBS for the project

3.4. Gantt Chart

A Gantt chart is a widely used project management tool that visually represents a project schedule by displaying tasks along a timeline. Tasks are listed vertically, while time is shown horizontally, with bars indicating each task's start date, end date, duration, progress, dependencies, milestones, and assigned team members. Gantt charts help teams easily understand what needs to be done, when it should be completed, and how tasks are interconnected (Canva, 2025).

3.4.1. Previous Gantt Chart

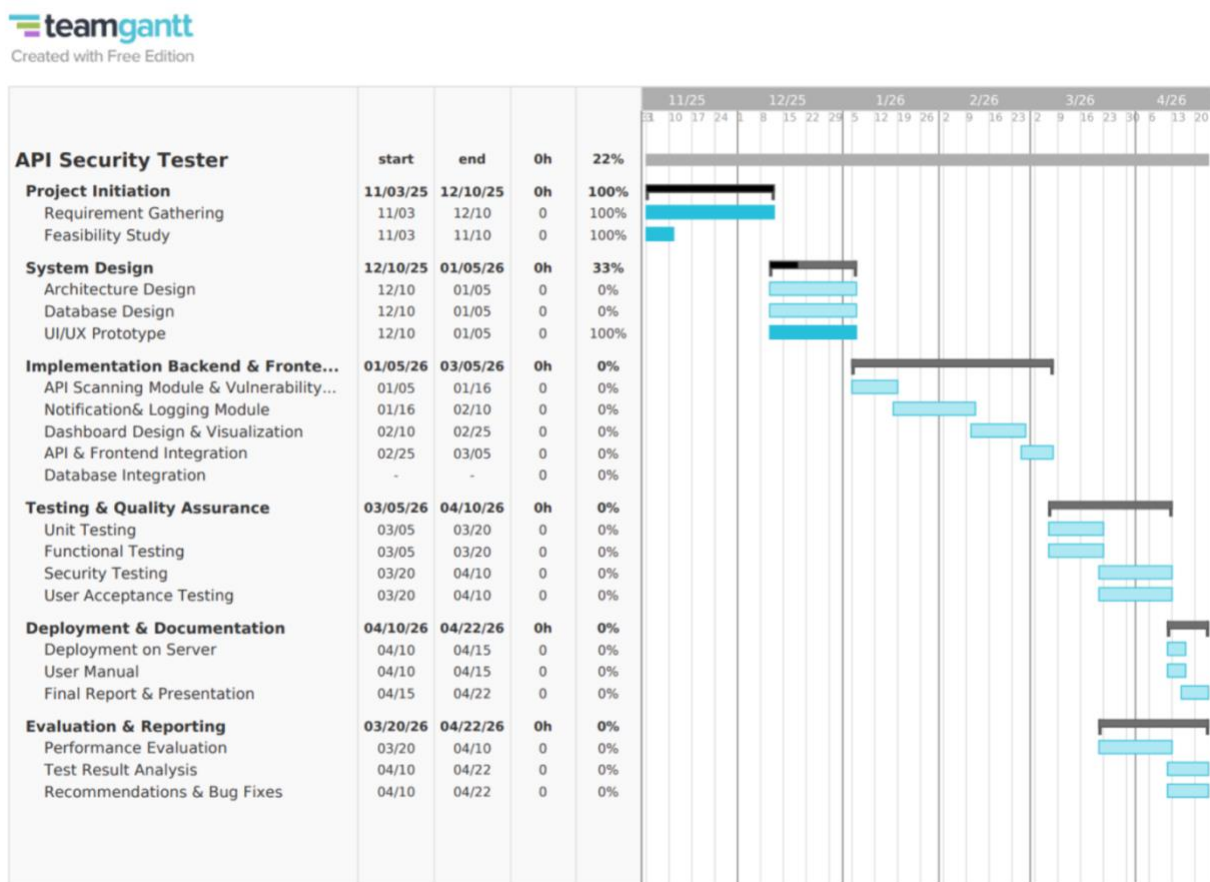


Figure 7 Gantt Chart

3.4.2. Updated Gantt Chart

The API Security Tester project adopts the Agile Incremental Framework with an initially suggested eight-week schedule segmented into separate increments. Nonetheless, the real execution has greatly sped up this timeline, as Increments 1-4 (Foundation, API Input, Security Testing, and Dashboard) were all finished in the initial two weeks rather than the intended five weeks. This swift advancement positioned the project three weeks in front of the timeline by December 19, 2025, reaching 55% completion with ten thorough security tests functioning, full database integration, and a user interface ready for production. The rapid progress stemmed from the efficient use of incremental methodology, well-defined requirements from the proposal stage, a modular architecture that allowed for concurrent development, and regular daily work sessions.

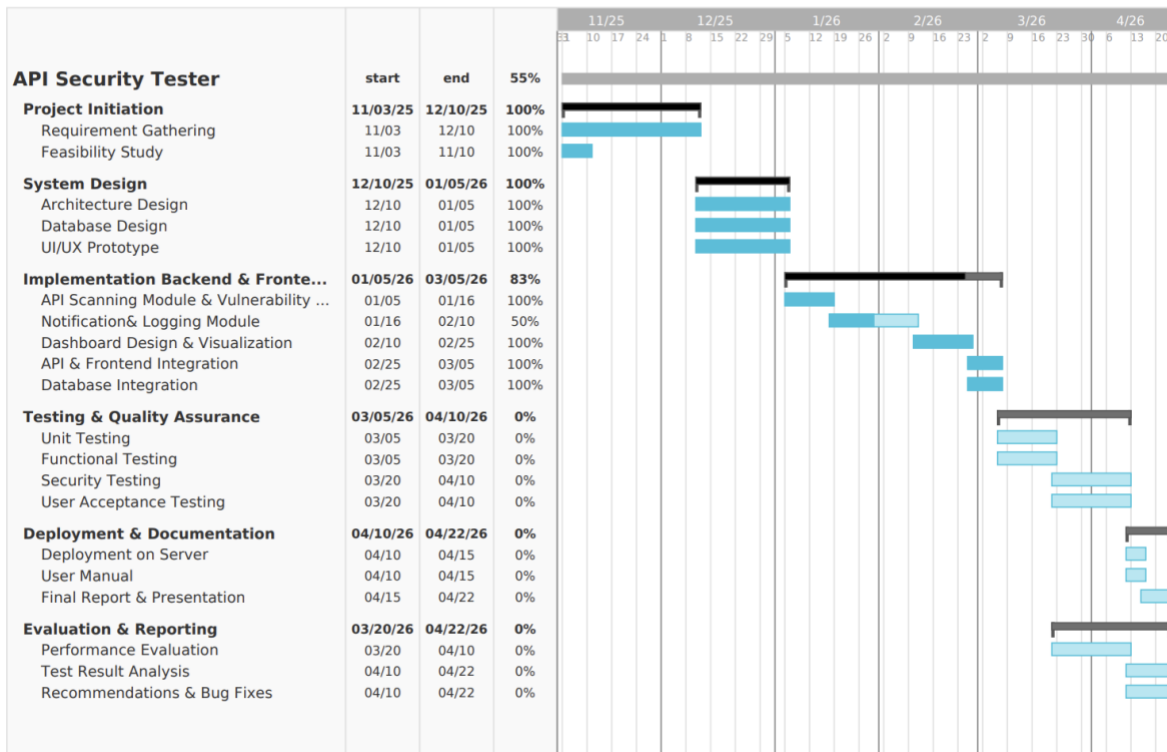


Figure 8 Updated gantt chart

3.5. Milestone

Read about milestones and the progress in [Appendix 6](#)

4. Development Till date

4.1. System Architecture Diagram

System architecture is the high-level design of the system that outlines the components of the system, how these components are structured, and how these various components interact to reach the high-level goals of the system. It outlines various system elements such as hardware and software so that it is able to function accordingly in the future (Aras, 2025).

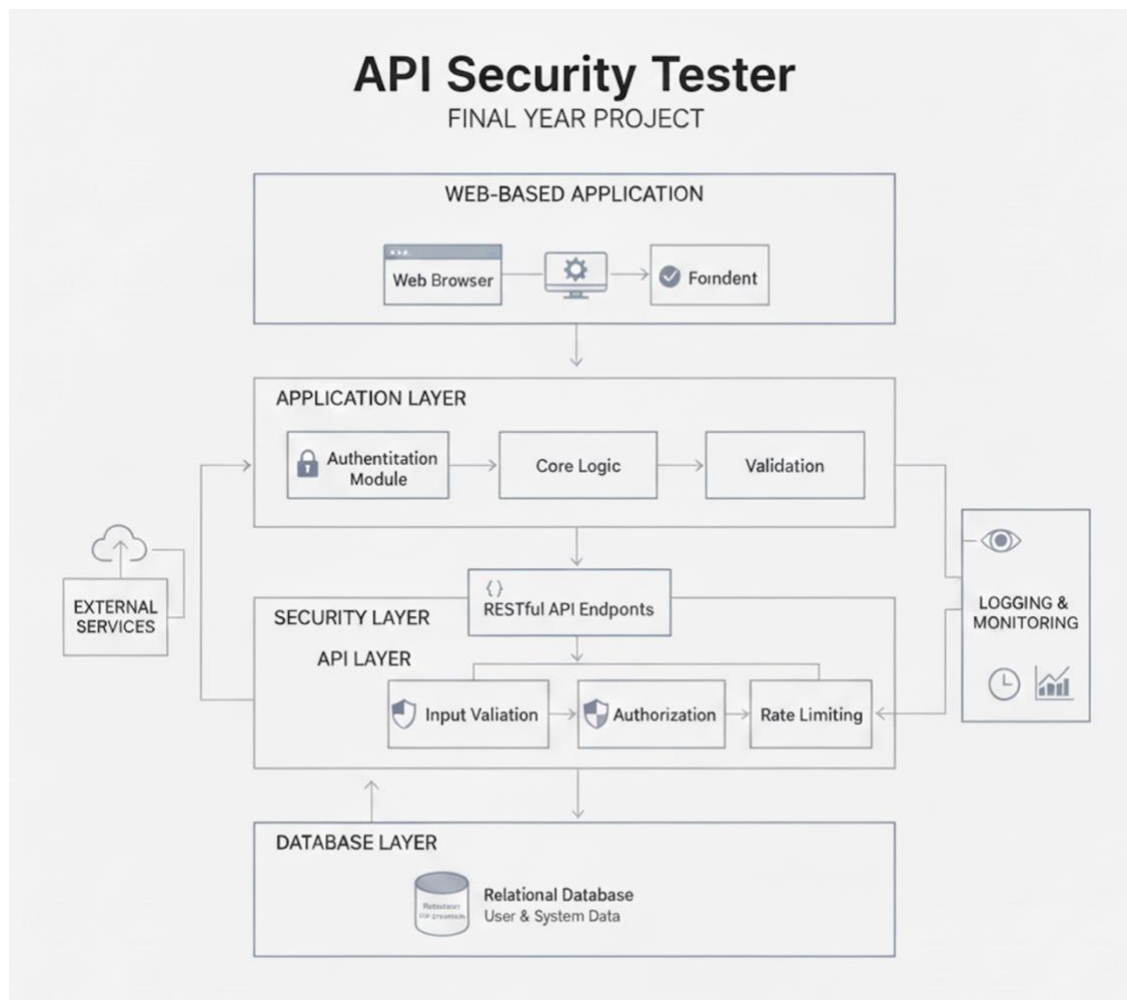


Figure 9 System Architecture Diagram

4.2. Use Case Diagram

A use case diagram (UML) visually represents various users (actors) who may utilize a system to complete their objectives. It's representative of what a system's functionality should provide from the user's standpoint (geeksforgeeks, 2025).

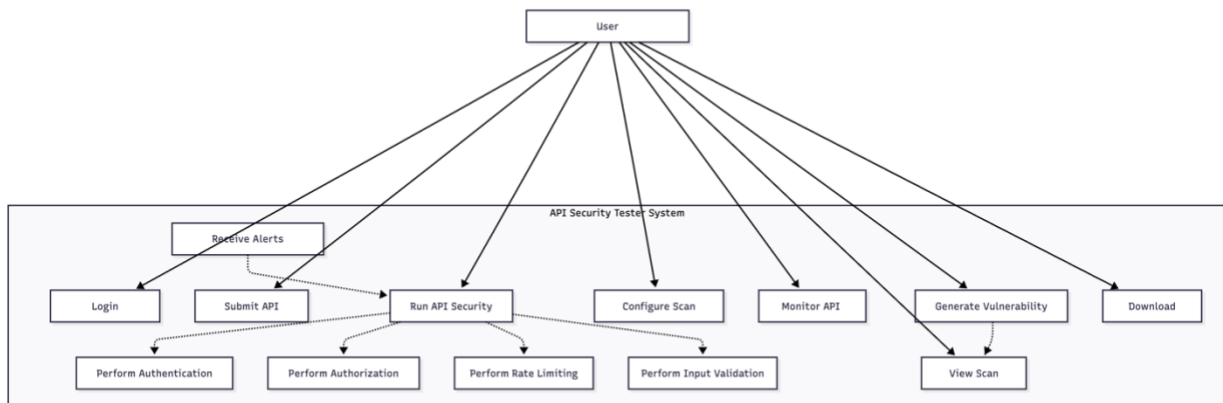


Figure 10 Use Case Diagram

4.3. Flowchart

A flowchart is a diagram that shows the steps and decisions in a process using connected symbols, in the order they happen (ASQ, 2025).

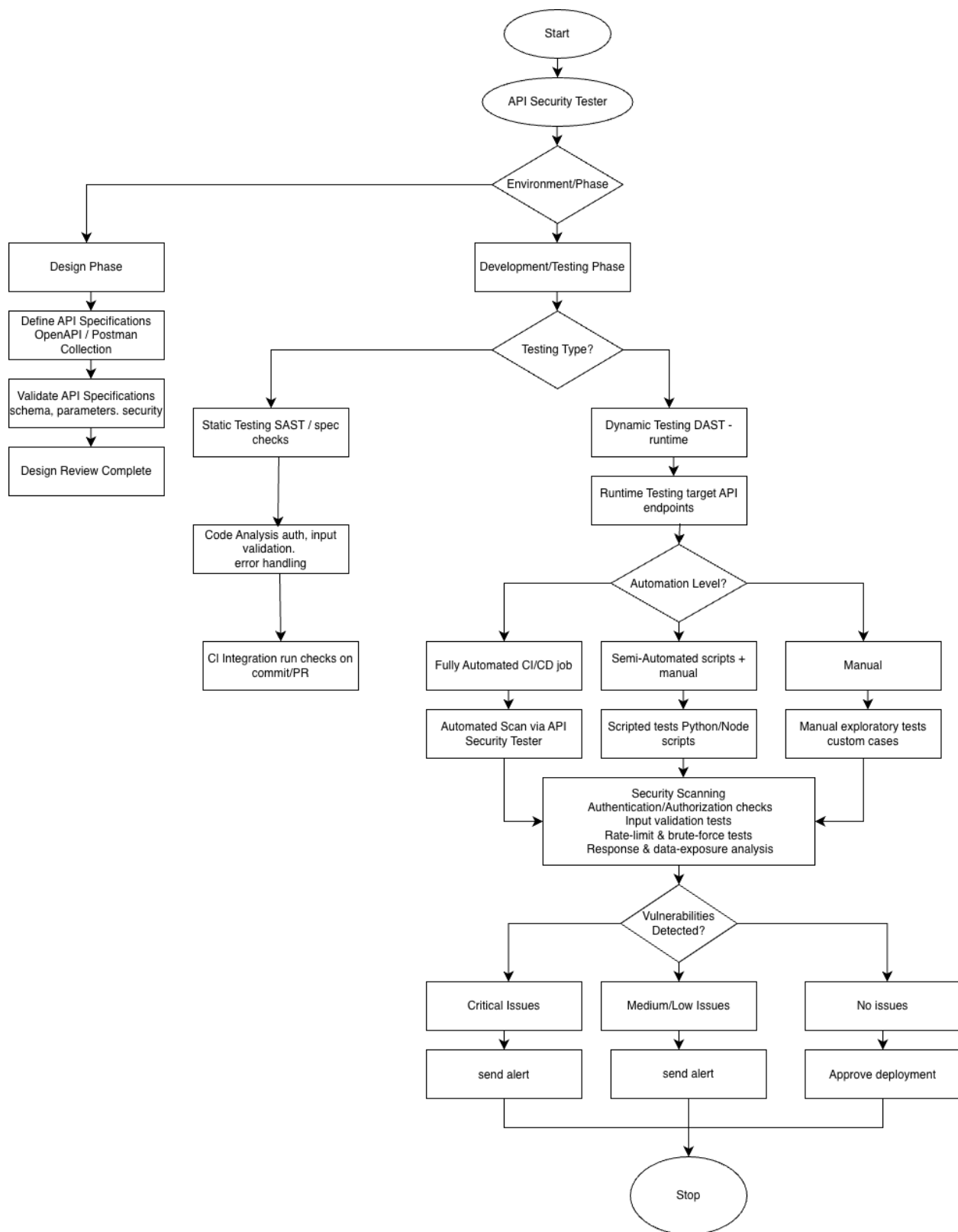


Figure 11 Flowchart

4.4. Wireframe

A wireframe represents a visual representation of the fundamental layout of your website or application (figma, 2025).

The wireframe depicts a web application titled "API Security Tester". At the top, there is a browser-like header with a tab labeled "A Web Page", navigation arrows, and a URL bar containing "https://". Below the header, the main content area is divided into two sections. The upper section, titled "Scan API Endpoint", contains four input fields labeled "API Name", "API URL", "HTTP Method", and "Description". Below these fields are two buttons: "Start Security Scan" and "Save Endpoint". The lower section, titled "Saved Api Endpoints", features a list area and a "Refresh" button. The footer of the application contains the text: "Developed by Samriddhi Poudel (23047345) Final Year Project 2025 | London Metropolitan University".

Figure 12 Wireframe

4.5. UI Design

API Security Tester v0.1.0 Disconnected

Automated API Vulnerability Scanner

Test your APIs for common security vulnerabilities based on OWASP Top 10

Scan API Endpoint

API Name (Optional)
e.g., GitHub API, Test API

API URL *
https://api.example.com/endpoint

HTTP Method
GET

Description (Optional)
Brief description of this API endpoint

Figure 13 UI Design

4.6. Development status with evidence

4.6.1. Development status

The creation of the API Security Tester and Vulnerability Scanner has now moved from design to implementation stages. The backend has been developed with config management, REST APIs, scanning logic, and database models, and the frontend has been implemented with a functional dashboard to interact with these APIs. A database schema and ERD has also been designed and implemented to create a system capable of analysing real scanned data to be stored in the system.

4.6.2. Evidence

Evidence 1: Project Structure

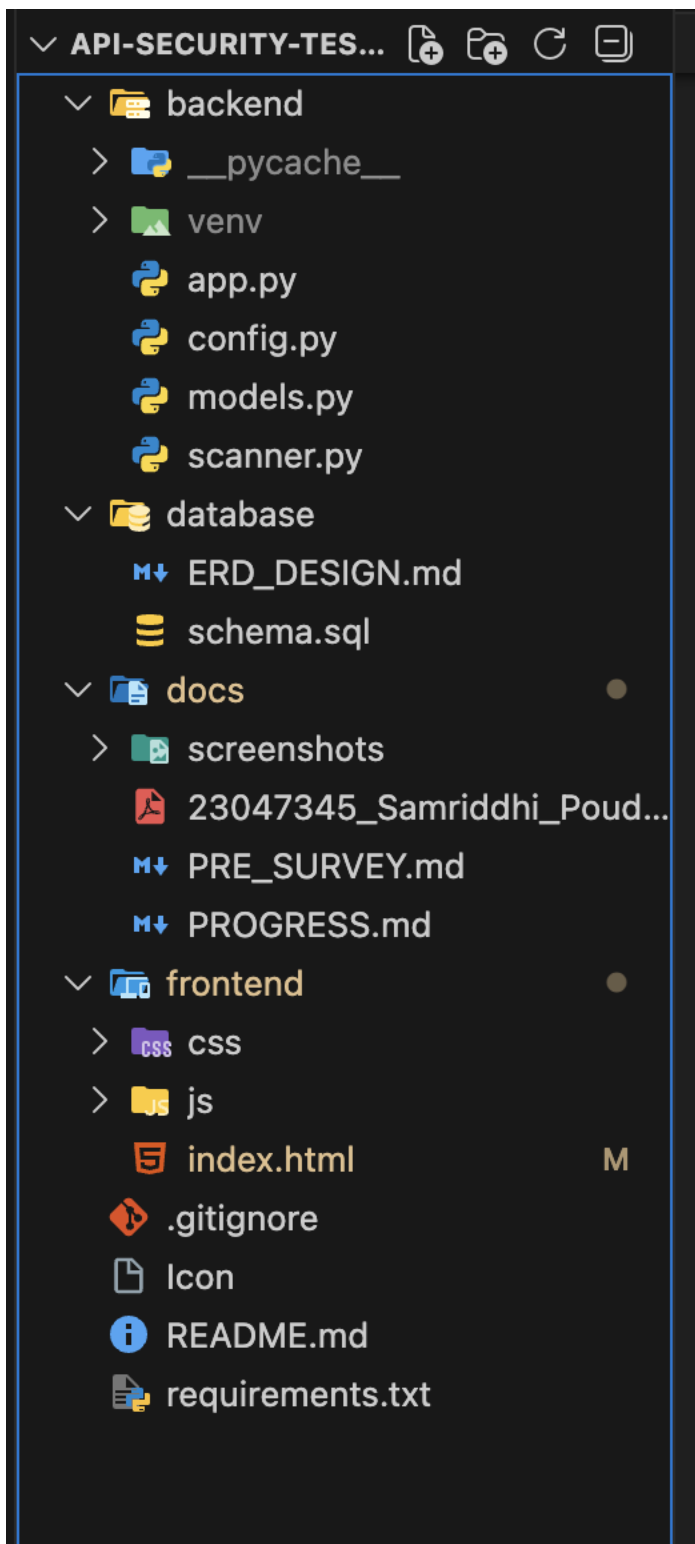


Figure 14 project structure in VS Code

Evidence 2: flask backend server running


```
inters-MacBook-Pro backend % python3 app.py
=====
API Security Tester v0.1.0
=====
Developer: Samriddhi Poudel (23047345)
Database: MySQL Connected
Server: http://localhost:8000
API Docs: http://localhost:8000/api/info
=====

Server starting...

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a p
roduction WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8000
* Running on http://100.64.215.99:8000
Press CTRL+C to quit
* Restarting with stat
=====
API Security Tester v0.1.0
=====
Developer: Samriddhi Poudel (23047345)
Database: MySQL Connected
Server: http://localhost:8000
API Docs: http://localhost:8000/api/info
=====

Server starting...

* Debugger is active!
* Debugger PIN: 104-867-216
```

Figure 15 Flask backend server running

Evidence 3: API Security tester scanning

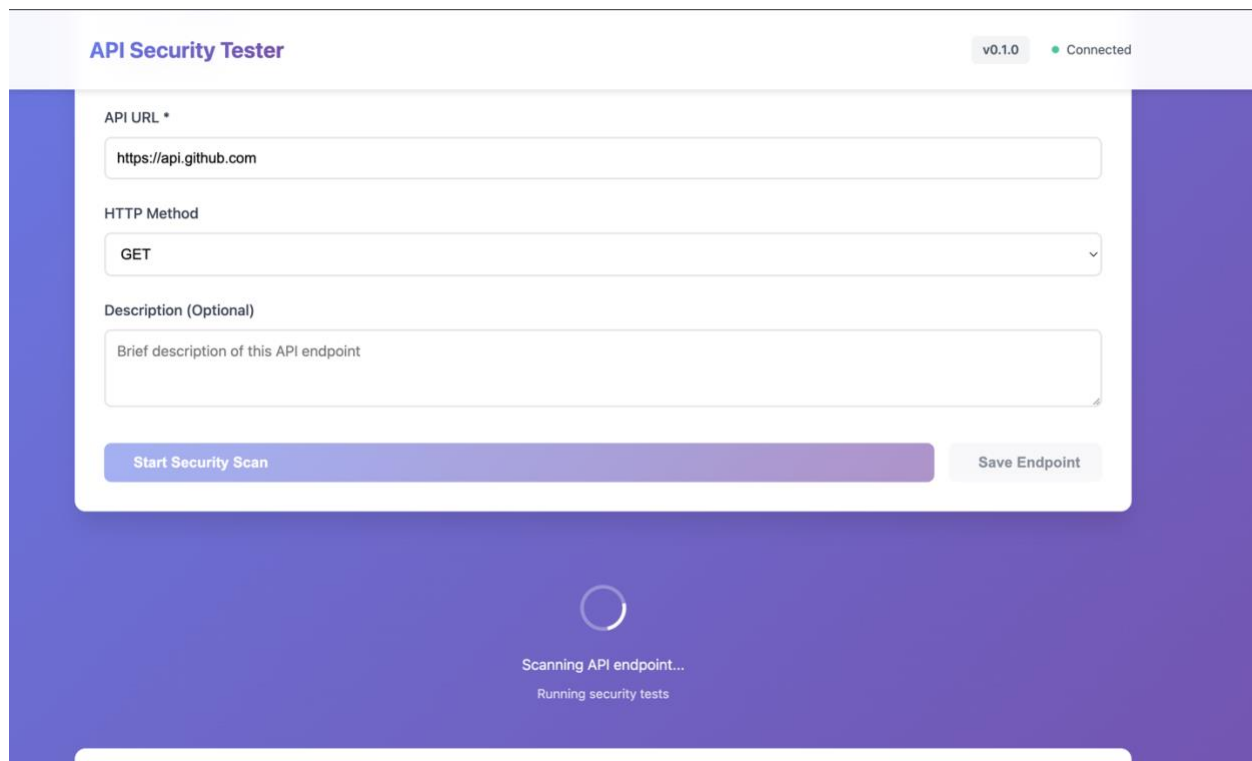
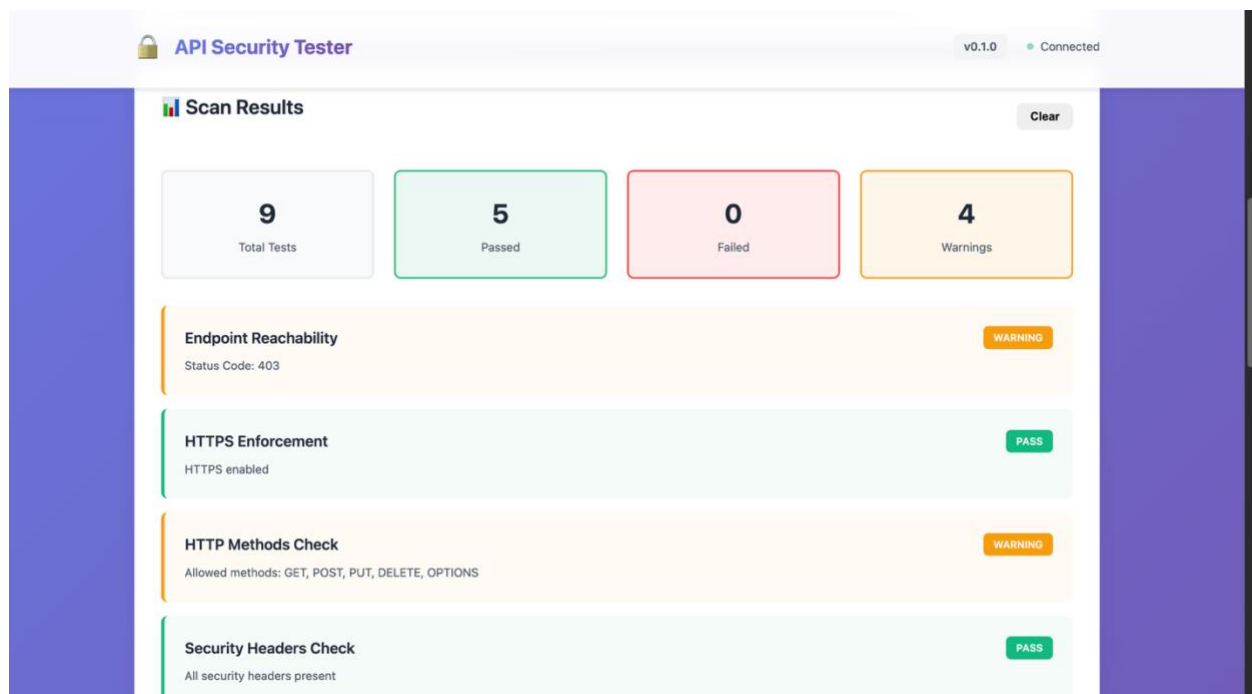


Figure 16 API security tester scanning api



Evidence 3: Databases

```

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_api_security_db |
+-----+
| alerts                    |
| api_endpoints              |
| scan_schedules             |
| scans                      |
| users                      |
| vulnerabilities            |
+-----+
6 rows in set (0.007 sec)

mysql> SELECT * FROM scans;
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | api_endpoint_id | scan_timestamp | status | total_tests | passed_tests | failed_tests | warnings | scan_duration |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 16 | 6 | 2025-12-22 04:05:51 | completed | 10 | 7 | 0 | 3 | NULL |
| 17 | 6 | 2025-12-22 11:39:58 | completed | 10 | 7 | 0 | 3 | NULL |
| 18 | 6 | 2025-12-28 04:03:44 | completed | 10 | 7 | 0 | 3 | NULL |
| 19 | 6 | 2025-12-28 07:36:32 | completed | 10 | 7 | 0 | 3 | NULL |
| 20 | 7 | 2025-12-28 07:37:39 | completed | 10 | 6 | 1 | 3 | NULL |
| 21 | 6 | 2025-12-30 07:16:48 | completed | 10 | 7 | 0 | 3 | NULL |
| 22 | 8 | 2025-12-30 07:43:52 | completed | 10 | 6 | 1 | 3 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
7 rows in set (0.008 sec)

mysql> SELECT * FROM vulnerabilities;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | scan_id | test_name | category | severity | status | details | recommendation | created_at |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 108 | 16 | Endpoint Reachability | General | PASS | PASS | Status Code: 200 | NULL | 2025-12-22 04:05:51 |
| 109 | 16 | HTTPS Enforcement | General | PASS | PASS | HTTPS enabled | NULL | 2025-12-22 04:05:51 |
| 110 | 16 | HTTP Methods Check | General | WARNING | WARNING | Allowed methods: GET, POST, PUT, DELETE, OPTIONS | NULL | 2025-12-22 04:05:51 |
| 111 | 16 | Security Headers Check | General | PASS | PASS | All security headers present | NULL | 2025-12-22 04:05:51 |
| 112 | 16 | Broken Authentication | General | WARNING | WARNING | Accessible without authentication | NULL | 2025-12-22 04:05:51 |
| 113 | 16 | SQL Injection Vulnerability | General | PASS | PASS | No SQL injection detected | NULL | 2025-12-22 04:05:51 |
| 114 | 16 | XSS Vulnerability | General | PASS | PASS | No XSS detected | NULL | 2025-12-22 04:05:51 |
| 115 | 16 | Rate Limiting | General | WARNING | WARNING | No rate limiting detected | NULL | 2025-12-22 04:05:51 |
| 116 | 16 | Excessive Data Exposure | General | PASS | PASS | No sensitive data exposure detected | NULL | 2025-12-22 04:05:51 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 17 Scan, vulnerability tables and sample data

```

mysql> SELECT * FROM api_endpoints;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | user_id | name | url | method | headers | body | description | created_at | updated_at |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 6 | NULL | GitHub API Test | https://api.github.com | GET | NULL | NULL | NULL | 2025-12-22 04:05:11 | 2025-12-22 04:05:11 |
| 7 | NULL | islington college | https://islingtoncollege.edu.np | GET | NULL | NULL | NULL | 2025-12-28 07:37:17 | 2025-12-28 07:37:17 |
| 8 | NULL | GitHub API Test | https://test-codert-vapt.netlify.app/ | GET | NULL | NULL | NULL | 2025-12-30 07:30:03 | 2025-12-30 07:30:03 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)

```

Figure 18 API endpoints stored in the database

5. Analysis of Progress

5.1. Progress Review

5.1.1. Current Scenario of Progress

The project is moving towards the right direction at the present stage as per the schedule. The project objectives and problem definition have been defined and the comprehensive literature regarding API security, vulnerabilities that are generally practiced, and the security testing tools that are currently available have been reviewed. System design and architecture in general have been delineated and important portions like request handling module, vulnerability detection modules and result reporting have been defined.

The basic development is underway and some of the first modules of the API security scanner are already operational, such as the basic request processing and the initial security checks. Preliminary testing has been conducted to test the functionality and the proper response of the API. The project is currently in the shift to the design stage to active development and testing and is currently oriented on the expansion of vulnerability checks, increased accuracy and performance.

In general, the project is within schedule, and significant steps have been made that can be measured and a clear roadmap used to finish off the remaining features and final evaluation.

5.1.2. Progress Timeline

The API Security Tester and Vulnerability Scanner is developed in an iterative manner. It involves feedback being provided continuously and improvement done during the process of development. Because of this circular thinking, some of the differences arose because of deviation of the initial plan. Consequently, the Gantt chart and Work Breakdown Structure (WBS) were updated and improved in terms of feedback and changing project needs. Although most activities were accomplished timely, some activities were delayed because of the lack of previous technical knowledge on advanced API security principles. Irrespective of these difficulties, the project has been moving in the right direction, and it will be completed successfully.

5.1.3. Action Plan

The rest project activities will be performed based on the revised Gantt chart based on incremental development model to maintain project schedule. The system will be built and released in successive phases where each phase will consist of requirement analysis, design, development, testing, and implementation. In addition to development, testing, system optimization and project documentation of each increment will be carried out to ascertain that all the parts are working and incorporating appropriately. Incremental feedback will be utilized in optimizing the next increment. By being dedicated to the accomplishment of the project within the allocated time and successfully handling the unanticipated problems, the milestone of the interim report project will be achieved successfully, and the project will proceed towards the final implementing and documenting phase.

6. Further Work

The remaining work for the API Security Tester and Vulnerability Scanner is planned in the form of three major stages: fulfillment of development cycles, extensive testing, and final optimization and documentation. These stages will be based on Agile Incremental Framework so that every increment will follow the requirement analysis, design, implementation, testing, and integration phase and then the final release will be done.

Development Phase:

The subsequent development tasks are expansion of the prototype beyond the ten initial testing and getting the system ready to be used in production. This would involve adding more OWASP API Security Top 10 tests (like more advanced authorization abuse and SSRF cases) and better handling of input data, as well as filling out the dashboard and visualization capabilities of the historical trends and severity-based display.

Testing Phase

After the basic functionality is done, a specific testing period will be used to ensure that everything is correct in terms of functionality, performance, and usability. Each vulnerability check will be verified through functional tests, and performance tests will also be used to test performance indicators such as scan time, concurrency, and database response time against the non-functional requirements (such as full scan in less than 30 seconds and fast load of UI).

The user-oriented testing will then be done based on the representative users like the developers, security engineers, and QA testers to get feedback on the readability of the result, dashboard friendly-use and usefulness of the report. They will provide feedback on how the interface, phrasing of vulnerability descriptions and presentation of recommendations to non-technical stakeholders can be improved.

Final adjustments and documentation

During the closing phase, all the problems that have been detected during testing and feedback will be resolved and stability and maintainability will be optimized in the system. This will entail refactoring of the code where needed, optimization of database queries, refining dashboard displays, and making sure that all modules work in harmony as one web-based solution.

A detailed record will be made simultaneously, including system design, installation procedures, user instructions, and the guidance on the optimal practices of scan results analysis. Such documentation along with the final reports will ensure that users and other stakeholders in the future can comprehend, implement and continue using API Security Tester and Vulnerability Scanner successfully.

7. Conclusion

The interim report has provided a strong foundation for the development of the API Security Tester and Vulnerability Scanner. The development is based on the need to respond to the increasing number of attacks related to APIs. The report has provided the problems, objectives, and expected results, which indicate that the new system will check for common vulnerabilities, keep tabs on the APIs that exhibit strange behaviour, produce alerts, and deliver detailed reports through an intuitive web interface. Through adopting the Agile Incremental approach, the project breaks down the activity into manageable chunks that are ideal for continuous input, constant testing, and responsiveness to new threats that may arise, which makes it more ideal for a project that deals primarily with issues of security.

The resource description, risk assessment, WBS, milestones, and Gantt chart employed also verify the realism of the project in terms of feasibility and that it can be accomplished within the proposed time. The project has been indicated to be on the right track towards developing a functional lightweight API security-testing tool that can improve the security position of an organization.

8. References

- Kukushkina, N., 2025. *Understanding the role of APIs in modern Software Development*. [Online]
Available at: [https://digitalmara.com/blog/understanding-the-role-of-apis-in-modern-software-development/#:~:text=APIs%20\(application%20programming%20interfaces\)%20serve,functions%2C%20key%20practices%20and%20trends.](https://digitalmara.com/blog/understanding-the-role-of-apis-in-modern-software-development/#:~:text=APIs%20(application%20programming%20interfaces)%20serve,functions%2C%20key%20practices%20and%20trends.)
- Penelope Todd, M. M. H. K. A. J., 2025. *Rate Limiting and Threat Detection in Intelligent API Gateways*, s.l.: Research Gate.
- Chinnasamy, V., 2023. *API Security 101: Understanding the Risks and Implementing Best Practices*. [Online]
Available at: <https://www.indusface.com/blog/what-is-api-security-and-why-is-it-important/>
- OWASP, 2023. *OWASP API Security Project*. [Online]
Available at: <https://owasp.org/www-project-api-security/>
[Accessed 2025].
- F5 Newsroom, 2025. *API Security Risks and Challenges*. [Online]
Available at: <https://www.f5.com/company/blog/api-security-risks-and-challenges>
[Accessed 2025].
- Kong, 2023. *API Infrastructure is Mission Critical — and Increasingly Under Attack*. [Online]
Available at: <https://konghq.com/blog/enterprise/apis-are-mission-critical>
[Accessed 2025].
- escape, 2024. *Escape analyzed 31 API Security Breaches*. [Online].
- Paulk, M. C., 2002. *Agile Methodologies and Process Discipline*, s.l.: crosstalk.
- geeksforgeeks, 2025. *Incremental Process Model - Software Engineering*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-incremental-process-model/>
- Raeburn, A., 2025. *The work breakdown structure (WBS) for project management: What it is and how to use it*. [Online]
Available at: <https://asana.com/resources/work-breakdown-structure>
[Accessed 2025].
- Canva, 2025. *Creating a Gantt chart: How-tos, templates, and tips*. [Online]
Available at: <https://www.canva.com/online-whiteboard/gantt-chart/>

- Gurnov, A., 2024. *What Is a Milestone in Project Management?*. [Online]
Available at: <https://www.wrike.com/project-management-guide/faq/what-is-a-milestone-in-project-management/>
[Accessed 2025].
- Singh, R., 2021. *Ways to Plan a Vulnerability Test Over a Web Application Using OWASP ZAP*. [Online]
Available at: <https://www.indusface.com/blog/ways-to-plan-a-vulnerability-test-over-a-web-application-using-owasp-zap/>
[Accessed 2025].
- Postman , 2025. *Organize and automate API requests in Postman Collections*. [Online]
Available at: <https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/>
[Accessed 2025].
- Portswigger, 2025. *Burp Suite Professional Features*. [Online]
Available at: <https://portswigger.net/burp/pro/features>
- Gliffy, 2020. *UML Diagrams: 14 Types and Templates*. [Online]
Available at: <https://www.gliffy.com/blog/uml-2-5-diagram-types-and-templates>
[Accessed 2025].
- Aras, 2025. *What Is System Architecture?*. [Online]
Available at: <https://aras.com/en/glossary/system-architecture>
[Accessed 2025].
- ASQ, 2025. *Flowchart*. [Online]
Available at: <https://asq.org/quality-resources/flowchart>
[Accessed 2025].
- geeksforgeeks, 2025. *Use Case Diagram - Unified Modeling Language (UML)*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/use-case-diagram/>
- figma, 2025. *What is wireframing?*. [Online]
Available at: <https://www.figma.com/resource-library/what-is-wireframing/>
- geeksforgeeks, 2025. *What is Spiral Model in Software Engineering?*. [Online]
Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-spiral-model/>
- Lewis, S., 2023. *prototyping model*. [Online]
Available at: <https://www.techtarget.com/searchcio/definition/Prototyping-Model>
[Accessed 2025].

9. Appendix

9.1. Appendix 1: Problem Scenario

(Problem areas identified)

- Authentication Failures: API may allow invalid or expired tokens, authenticate without proper session management, and may not have multi-factor authentication of sensitive operations.
- Bypasses of Authorization: Broken object-level and function-level authorization allow users to access resources or perform actions beyond their permission levels.
- Injection Vulnerabilities: SQL injection, NoSQL injection and command injection are still common in the areas where APIs cannot validate or sanitize user input.
- Data Exposure: API responses often include unnecessary data and reveal sensitive information (e.g. passwords, tokens, PII, internal system information).
- Rate Limiting Lack: API is prone to brute force attacks, denial of service and resource consuming without appropriate throttling.
- Security Compromise: Lack of security header and verbose error messages and inappropriate CORS configurations are vulnerabilities that can be exploited.
- SSRF Vulnerabilities: API endpoint programs that retrieve remote assets without checks can be led into accessing an internal system.

[Click here](#) to go back to top.

9.2. Appendix 2: Client's Requirements

Functional Requirements:

FR1: Vulnerability Detection

- The system should identify 8 or above of the OWASP API Security Top 10 vulnerabilities.
- Should be able to test breaks in authentication, malfunctioning authorization, data overexposure, security settings, SSRF, and rate limiting challenges.

FR2: API Endpoint Management

- The consumers should be able to plug API endpoints through web interface.
- System is supposed to handle various HTTP (GET, POST, DELETE) methods.
- Should be able to save and manage a variety of API endpoints to be tested repeatedly.

FR3: Automated Security Testing

- System should perform full security scans automatically.
- Should be able to scan full vulnerability in 30 seconds.
- Should be able to address API failures without crashing the system.
- Should be able to support HTTP and HTTPS protocols.

FR4: Results Visualization

- Should be able to show test results as they are generated.
- Should have color-coded pointers on the level of severity (Pass/Fail/Warning).
- Should demonstrate overall statistics (number of tests, number of successful results, number of unsuccessful results, number of warnings)

FR5: Historical Data Management

- All scan results should be stored in persistent database in system.
- The users should be able to see scan history.

FR6: Monitoring and Alerting

- Support planned automatic scan.
- Offer real-time monitoring dashboard charts.

Non-Functional Requirements:**NFR1: Usability**

- The interface should be easy to use by non-technical users.
- Should give proper error messages and instructions.
- Should be available through web browser and non-installation.

NFR2: Performance

- Whole body scan must take less than 30 seconds.
- System should be able to test several APIs simultaneously.
- The response of database queries should take less than 2 seconds.
- Web interface must load within less than 3 seconds.

NFR3: Compatibility

- Should support Windows and MacOS, and Linux.
- Should support Chrome, Firefox, Safari, and Edge browsers.

[Click here](#) to go back to top

9.3. Appendix 3: Spiral Methodology

The spiral model is a risk-oriented software development framework that structures tasks into recurring cycles (spirals), with each cycle progressively enhancing the system while managing uncertainty. In each spiral, the team initially sets the objectives, then identifies and assesses significant risks, creates and tests a version of the system to mitigate those risks, and ultimately evaluates the outcomes with stakeholders before determining the next steps for the subsequent iteration (geeksforgeeks, 2025).

This framework renders the spiral model particularly ideal for intricate or high-risk projects, as it facilitates early identification of issues, ongoing refinement of requirements, and gradual enhancement of the solution rather than adhering to a fixed design from the outset (geeksforgeeks, 2025).

[Click here to return to top](#)

9.4. Appendix 4: Prototype Methodology

The prototype methodology (prototyping model) is a software development technique in which an initial working model of the system is created, presented to users, and continuously improved until it aligns with their requirements. It begins with collecting fundamental requirements and developing a fast, streamlined version of the app that emphasizes essential screens and main functions instead of complete logic or efficiency. Users test this prototype, provide input on its strengths and weaknesses, and the development team refines the prototype through multiple iterations before finalizing the complete implementation, which aids in clarifying ambiguous requirements, enhancing usability, and minimizing the risk of creating an incorrect system (Lewis, 2023).

[Click here to return to top](#)

9.5. Appendix 5: Agile Methodology

Agile methodology is a disciplined yet adaptable approach best suited for projects with evolving requirements and active customer collaboration. When applied appropriately, it supports efficient development while maintaining quality and responsiveness.

Key principles of Agile Methodology

- Focuses on customer satisfaction through early and continuous software delivery
- Welcomes changing requirements, even in late development stages
- Delivers software in short, frequent iterations
- Encourages close collaboration between developers and customers
- Values working software as the main measure of progress
- Promotes self-organizing and motivated teams

Core Characteristics

- Iterative and incremental development Small, cross-functional teams
- Continuous testing and integration
- Emphasis on technical excellence and simplicity

Advantages of Agile

- Faster delivery of usable software
- Better adaptability to requirement changes
- Improved customer involvement and satisfaction
- Higher product quality through continuous feedback Reduced project risk

[Click here to return to top](#)

9.5.1. Incremental Framework

Phases of the Incremental Framework

Each increment follows these phases repeatedly until the final system is completed:

Requirement Analysis

- Requirements are gathered and analyzed for the current increment.
- Focuses on defining only the features to be developed in that increment.
- Provides a clear foundation for design and development.

Design & Development

- The system architecture and components are designed for the selected features.
- Coding is done to implement the required functionality.
- Each increment builds on the previous version of the system.

Testing

- Newly developed features are tested for correctness and performance.
- Existing features are also tested to ensure no errors are introduced.
- Ensures reliability and quality of each increment.

Implementation

- The tested increment is deployed or integrated into the existing system.
- The system becomes more complete with every release.
- The process repeats until the final product is ready.

Key Characteristics of the Incremental Framework

- **Step-by-step development:** The system is built gradually rather than all at once.
- **Partial system delivery:** Each increment delivers a working part of the software.
- **Early availability of functionality:** Basic features are available early in the development cycle.
- **Customer feedback:** Feedback is collected after every increment to improve future releases.

- **Flexibility to change:** New requirements or changes can be introduced between increments.
- **Combination of models:** Merges the structured approach of the Waterfall model with iterative development (geeksforgeeks, 2025).

[Click here](#) to go back to top.

9.6. Appendix 6: Milestones

A milestone is a specific point within a project's life cycle used to measure the progress toward the goal (Gurnov, 2024). The following is the milestone for my project.

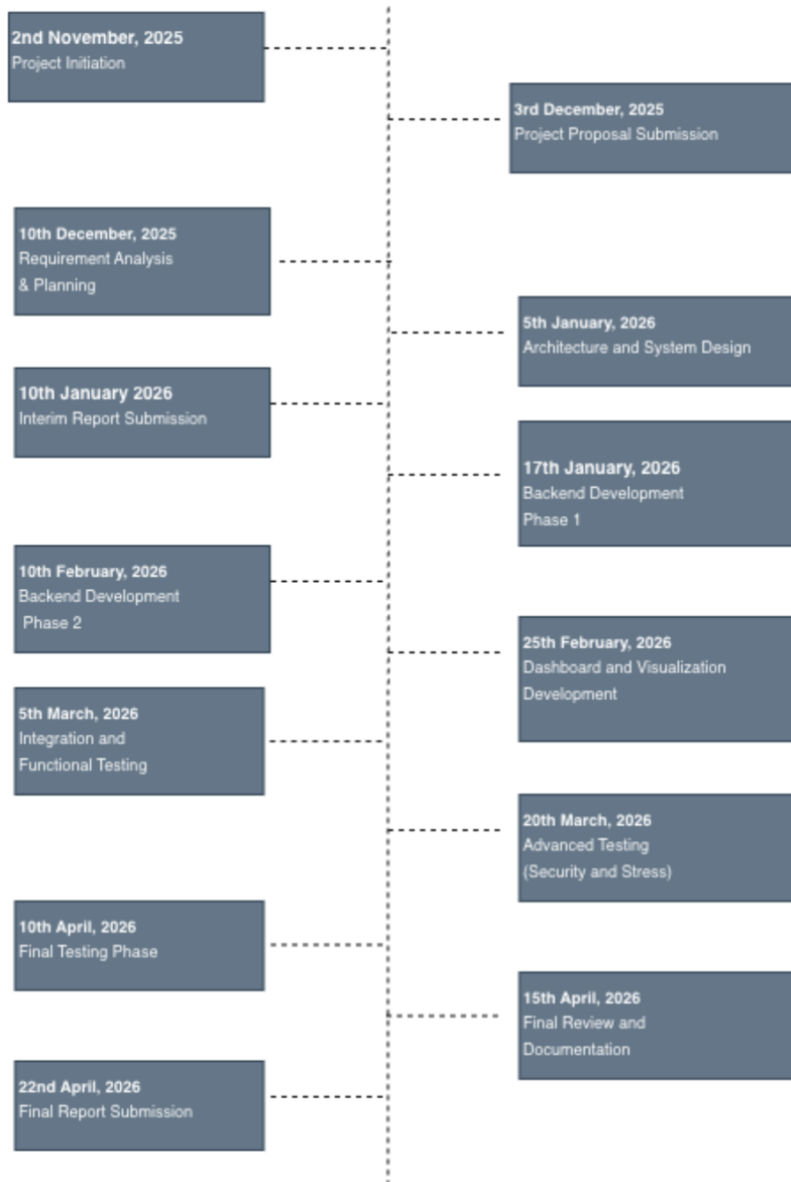


Figure 19 Milestone

1.1.1. Progress of milestones

The API Security Tester project is progressing significantly ahead of schedule. The initial milestones including Project Initiation, Project Proposal Submission, Requirement Analysis & Planning, Architecture and System Design, and Backend Development Phase 1 are all completed, reflecting exceptional project progress. Currently, the Interim Report Submission is in progress with substantial completion achieved. The remaining tasks including Backend Development Phase 2, Dashboard and Visualization Development, Integration and Functional Testing, Advanced Testing for Security and Stress, Final Testing Phase, Final Review and Documentation, and Final Report Submission are yet to be completed but are well-planned with clear timelines. The project is currently three weeks ahead of the original schedule with 55% completion achieved, which will ensure successful implementation through systematic development, comprehensive testing phases, and thorough documentation of each milestone.

Progress Table

Milestones	Completion Date	Status
Project Initiation	2 nd November, 2025	Completed
Project Proposal Submission	3 rd December, 2025	Completed
Requirement Analysis & Planning	10 th December, 2025	Completed
Architecture and System Design	5 th January, 2025	Completed
Interim Report Submission	10 th January, 2025	In progress
Backend Development Phase 1	17 th January, 2025	Completed
Backend Development Phase 2	10 th February, 2025	In progress
Dashboard and Visualization Development	25 th February, 2025	Completed
Integration and Functional Testing	5 th March, 2025	To do
Advanced Testing (Security and Stress)	20 th March, 2025	To do
Final Testing Phase	10 th April, 2025	To do
Final Review and Documentation	15 th April, 2025	To do
Final Report Submission	22 nd April, 2025	To do

Table 2 Progress table of milestones

[Click here](#) to go back to top

9.7. Appendix 8: Findings of Pre-Survey

Pre-survey form link : <https://forms.gle/nUSR5Fcq9eBboJ5Y7>

1. What is your current role?

37 responses

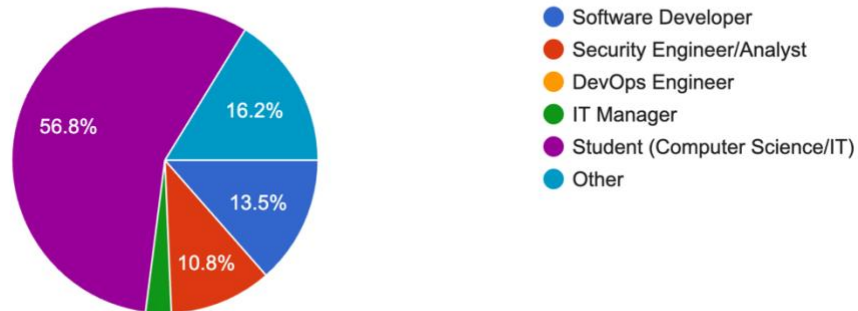


Figure 20 Pre-survey finding 1

2. How many years of experience do you have in software development or IT?

37 responses

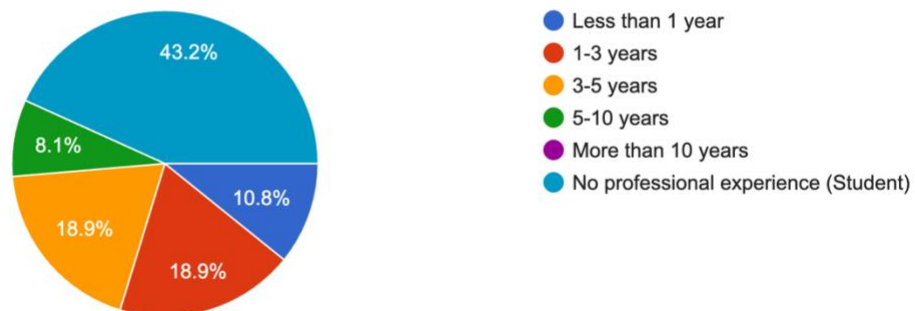


Figure 21 Pre-survey finding 2

3. Are you familiar with APIs (Application Programming Interfaces)?

37 responses

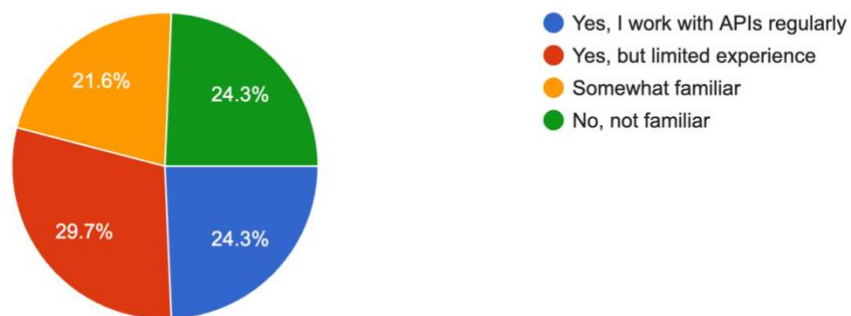


Figure 22 Pre-survey finding 3

4. How important do you think API security is for modern applications?

37 responses

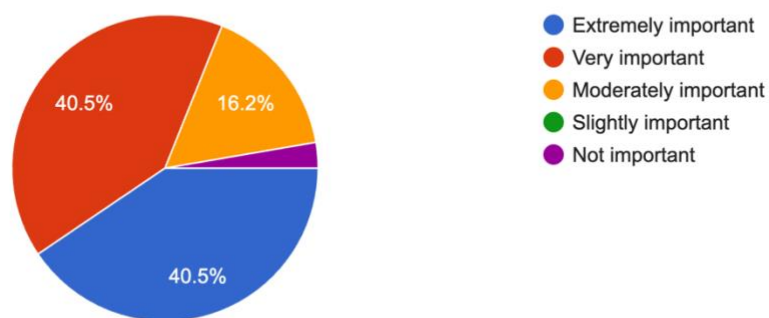


Figure 23 Pre-survey finding 4

5. Have you or your organization experienced any API security incidents? (data breach, unauthorized access, etc.)

37 responses

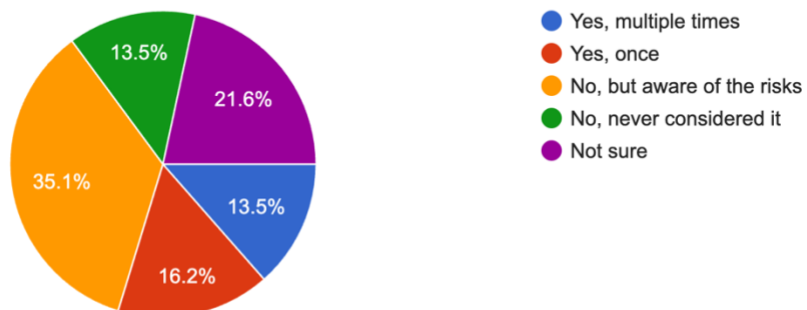


Figure 24 Pre-survey finding 5

6. Which of the following API security vulnerabilities are you aware of? (Select all that apply)

37 responses

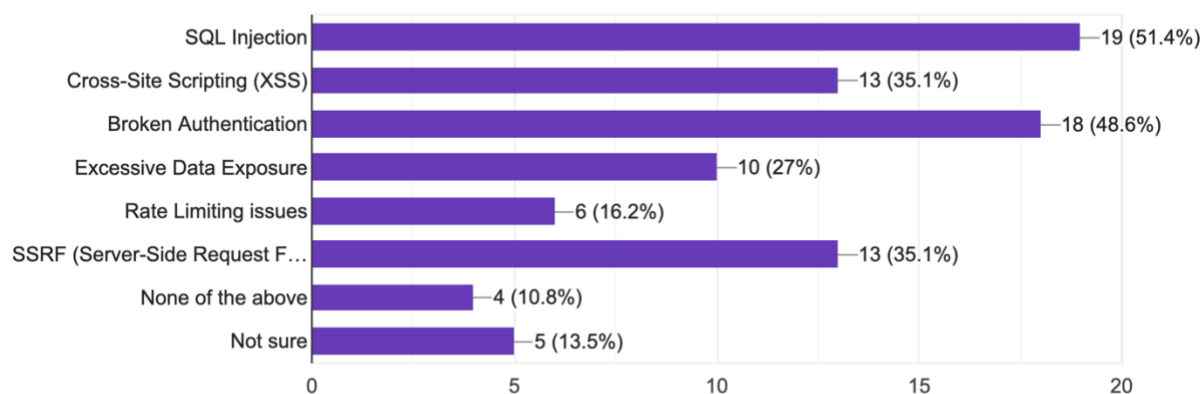
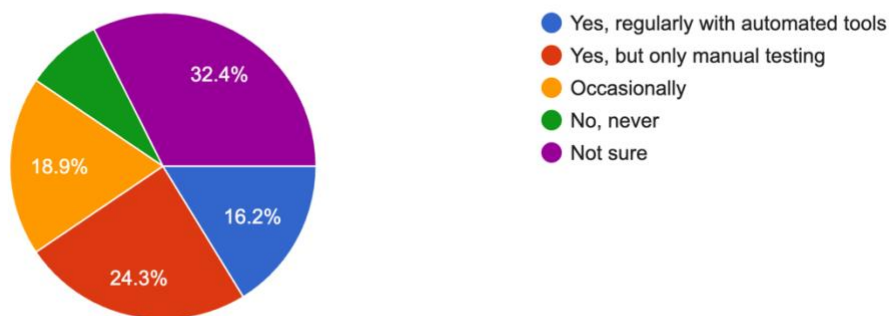


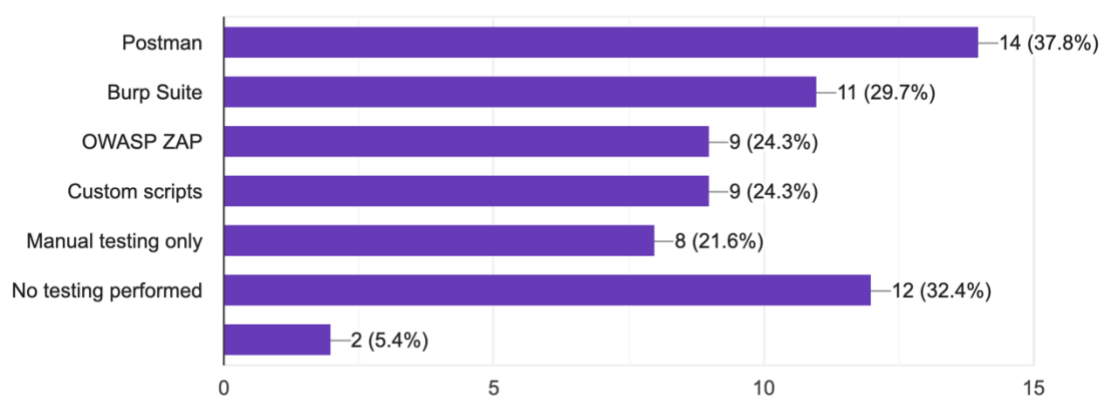
Figure 25 Pre-survey finding 6

7. Does your organization/project currently perform API security testing?

37 responses

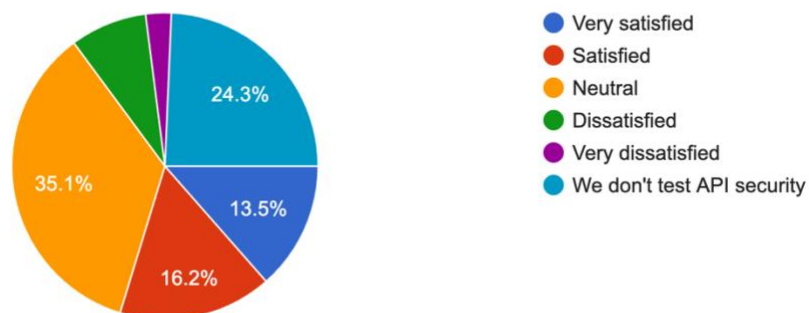
*Figure 26 Pre-survey finding 7***8. What tools or methods do you currently use for API security testing? (Select all that apply)**

37 responses

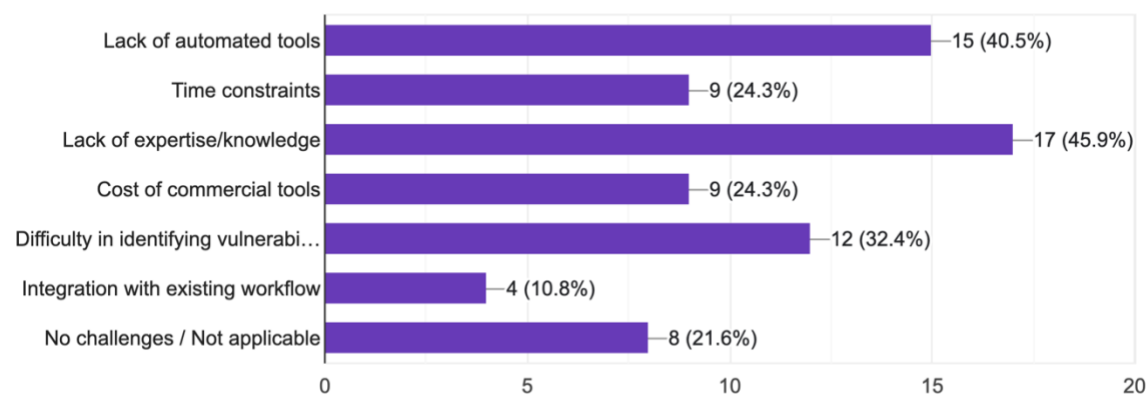
*Figure 27 Pre-survey finding 8*

9. How satisfied are you with your current API security testing approach?

37 responses

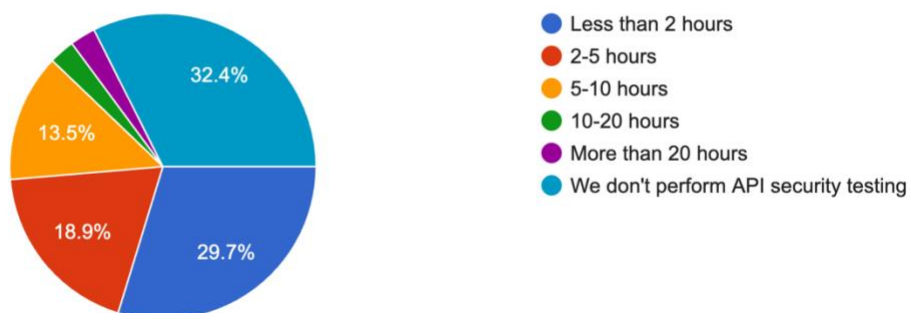
*Figure 28 Pre-survey finding 9***10. What are the biggest challenges you face in API security testing? (Select all that apply)**

37 responses

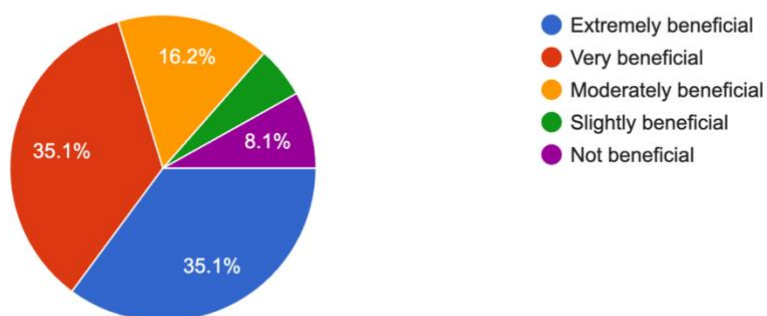
*Figure 29 Pre-survey finding 10*

11. How much time does your team typically spend on API security testing per project?

37 responses

*Figure 30 Pre-survey finding 11***12. Would an automated API security testing tool be beneficial for your work/organization?**

37 responses

*Figure 31 Pre-survey finding 12*

13. Which features would you consider most important in an API security testing tool? (Select all that you consider important)

37 responses

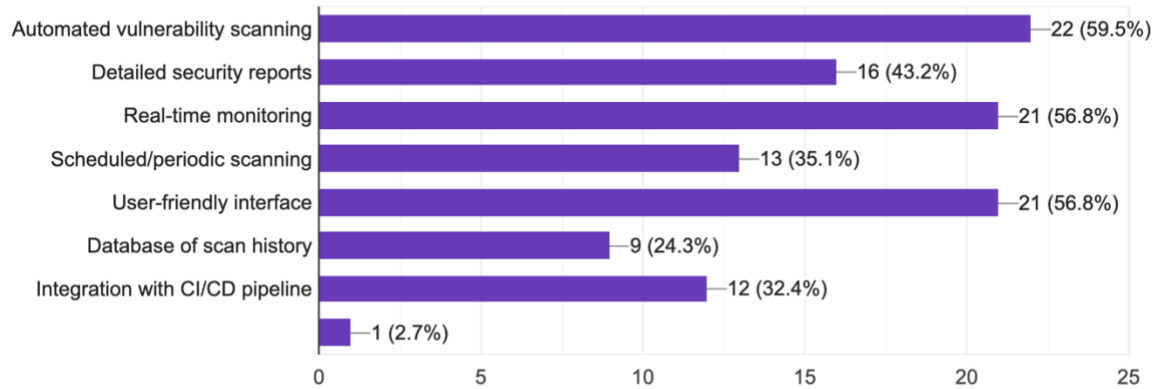


Figure 32 Pre-survey finding 13

14. What type of vulnerabilities would you most want an automated tool to detect? (Select top 3)

37 responses

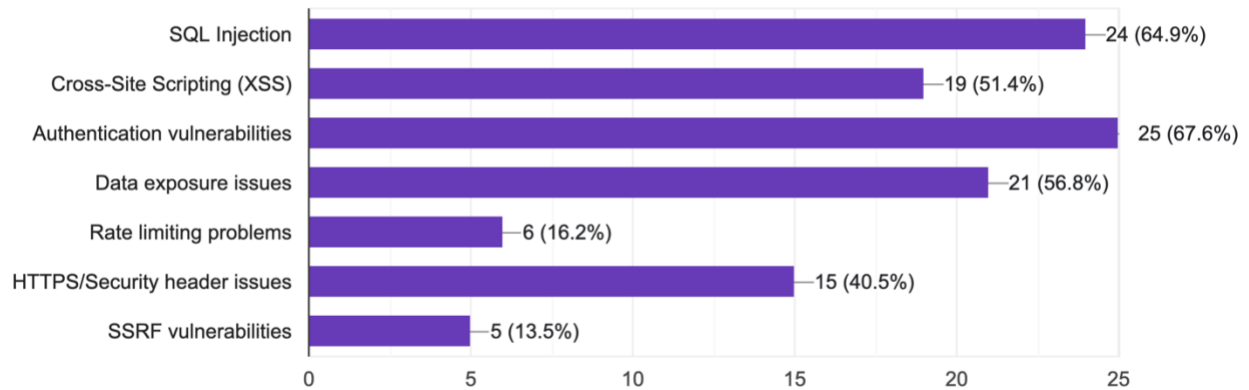


Figure 33 Pre-survey finding 14

15. Which of the following API security vulnerabilities have you encountered or are most concerned about? (Select all that apply)

36 responses

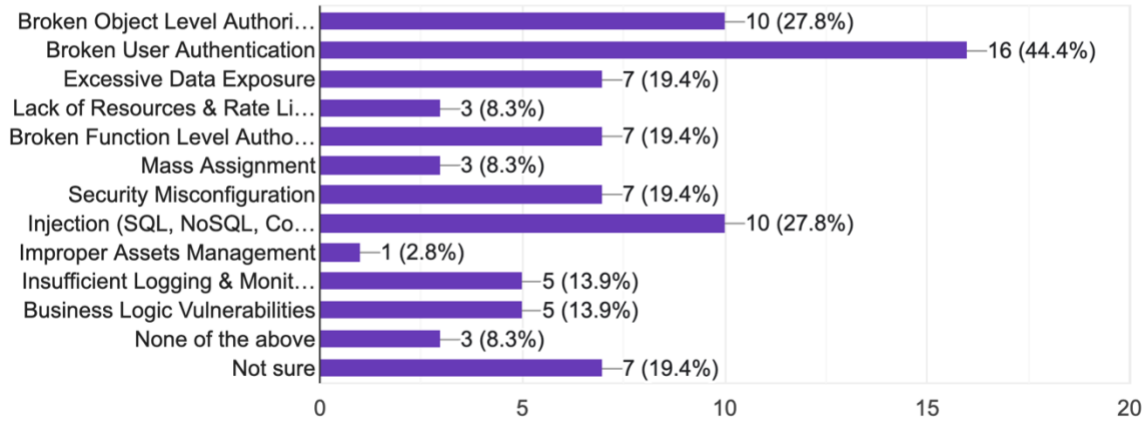


Figure 34 Pre-survey finding 15

16. If a free, open-source API security testing tool was available, how likely would you be to use it?

37 responses

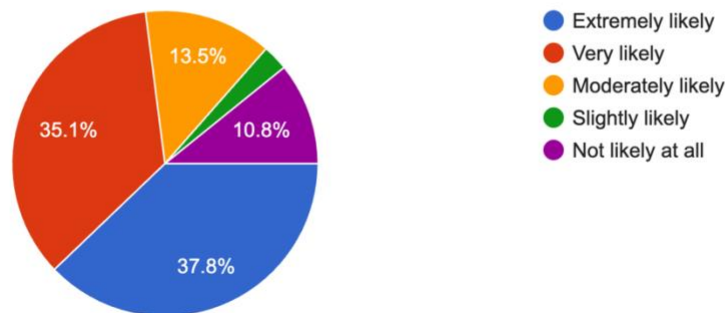


Figure 35 Pre-survey finding 16

The pre-survey included 37 participants comprising software developers, security engineers, DevOps engineers, IT managers, and students with computer science and IT expertise. This composition suggests that the recommended solution should cater to both technically skilled practitioners and early-career or student users, ensuring a clear and understandable presentation of security results.

The replies indicate a diverse array of experience levels, with a notable percentage of respondents possessing between one and five years of experience in software development or IT. Often, API testing efforts emphasize functional validation with tools like Postman, whereas specific API security testing is seldom conducted or overlooked entirely, primarily because of time limitations and insufficient security knowledge. Participants showed significant enthusiasm for automated checks that adhere to the OWASP API Security Top 10, as well as for succinct dashboards, past scan results, and straightforward remediation advice.

These results reinforce the necessity for a unified API Security Tester and Vulnerability Scanner that merges automated security evaluations with clear reporting, allowing teams to enhance their API security stance without needing expert penetration-testing expertise.

[Click here](#) to go back to the first part of this report.